Optimization Framework for Multi-Goal Terrain Navigation with Fuel-Constrained Path Planning

Dhanya [1] **Harsha** [2] **Suriyaa** [2]

Abstract

The project introduces a Deep Reinforcement Learning (DRL) approach using Proximal Policy Optimization (PPO) for fuel-constrained, multi-goal terrain navigation. The challenge is modeled as a Markov Decision Process (MDP) in procedurally generated 3D environments, which feature complex elevation maps (Perlin noise). The agent learns a policy to efficiently collect all goals while adhering to a limited fuel budget that depletes with movement and penalizes uphill travel. The PPO agent uses a shared Actor-Critic network and shows stable policy convergence and a consistent improvement in cumulative reward over training iterations.

The complete implementation and source code are available at: https://github.com/SuriyaaMM/stnav.

1. Introduction

Autonomous navigation in challenging terrains represents a fundamental problem in robotics and artificial intelligence (Thrun et al., 2005), with applications ranging from planetary exploration to search and rescue operations. Traditional path planning approaches, such as A* (Hart et al., 1968), often struggle with the complexity of real-world terrains and the need to balance multiple competing objectives such as energy efficiency, goal achievement, and environmental uncertainty.

Recent advances in deep reinforcement learning (DRL) (Mnih et al., 2015) have shown promising results for navigation tasks in complex environments. However, most existing approaches focus on single-goal navigation (Zhu et al., 2017; Mirowski et al., 2016) or operate in simplified 2D environments. The challenge of multi-goal collection with resource constraints in 3D procedurally generated terrains remains largely unexplored.

[*]Equal contribution [1]Department of Computer Science Engineering [2]Department of Electrical Engineering. Correspondence to: Dhanya <cs23b055@iittp.ac.in>, Harsha <ee23b015@iittp.ac.in>, Suriyaa <ee23b054@iittp.ac.in>.

CS515L Computational Methods for Optimization, Department of Computer Science, IIT Tirupati

2. Problem Statement

The goal is to navigate an agent across a discrete 2D terrain grid $h : \mathbb{Z}^2 \to \mathbb{R}$ while visiting a set of goals $\mathcal{G}$ as efficiently as possible. The task is to find a trajectory $\{(x_t, y_t)\}_{t=0}^{T}$ that:

- **Minimizes fuel consumption**, including elevation-dependent costs,
- **Satisfies movement constraints** (one grid step per action),
- **Remains within a fixed fuel budget**.

We assume a discrete grid representation for computational tractability, and that locomotion, particularly changes in elevation constitutes the dominant contributor to fuel usage. The problem is therefore a constrained optimization task over terrain with varying elevation.

2.1. Potential Solutions

Several modelling approaches can be considered for solving the fuel constrained terrain navigation problem.

Graph-Based Search (A*): The grid can be formulated as a graph with nodes (x, y) and edge costs defined by the fuel model:

$$\text{cost}(s_{t-1}, s_t) = F_{\text{base}} + F_{\text{climb}}(s_{t-1}, s_t).$$

A* can then be used to compute the least cost path. However, it must recompute the solution for every new start, goal set, or terrain realization, and does not scale to multi-goal scenarios where visiting all goals requires evaluating up to $N!$ orderings.

Dynamic Programming Approach A Dynamic Programming (DP) formulation considers a state of the form

$$X_t = (s_t,\ f_t,\ \mathcal{G}_t),$$

where s_t is the position, f_t is remaining fuel, and $\mathcal{G}_t$ is the set of visited goals.

The main limitation is the size of the DP state space:

$$|S| \times |F| \times 2^{|\mathcal{G}|},$$

which grows exponentially because of the 2^N possible goal-visit combinations. Even small grids with a few goals become computationally infeasible.

Thus, while DP is exact in principle, it is not practical for large terrains or multi-goal navigation.

Learning-Based Methods: Reinforcement learning provides an alternative by learning a policy rather than explicitly computing optimal values for every state. A neural network serves as a function approximator for the policy, enabling generalization across start positions, goals, and new terrain realizations. RL also naturally handles stochasticity and large state spaces, making it a scalable solution for complex, fuel-constrained navigation tasks.

3. RL Model Formulation

3.1. Markov Decision Process Formulation

The terrain navigation problem is formulated as a Markov Decision Process (MDP), corresponding to a discrete-time Markov chain parameterized by an agent policy. The MDP is defined as the tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma),$$

3.2. State Space Formulation

The agent's state $s_t \in \mathcal{S}$ at time step t is represented as a stack of four feature maps with flexible spatial dimensions:

$$s_t = [M_{\text{terrain}}, M_{\text{goal}}, M_{\text{agent}}, M_{\text{fuel}}]^T \in \mathbb{R}^{4\times H\times W} \quad (1)$$

where H and W represent the height and width of the terrain grid, which can be configured based on the specific environment requirements. Each component is defined as follows:

Terrain Map ($M_{\text{terrain}} \in \mathbb{R}^{H\times W}$): A continuous elevation map generated using Perlin noise (Perlin, 1985) to ensure procedural terrain variation. Each pixel (i, j) contains the normalized elevation value $h_{i,j} \in [0, 1]$.

Goal Map ($M_{\text{goal}} \in \{0, 1\}^{H\times W}$): A binary mask indicating goal locations, where $M_{\text{goal}}(i, j) = 1$ if position (i, j) contains an unvisited goal, and 0 otherwise.

Agent Position Map ($M_{\text{agent}} \in \{0, 1\}^{H\times W}$): A one-hot encoding of the agent's current position (x_t, y_t), where $M_{\text{agent}}(i, j) = 1$ if $(i, j) = (x_t, y_t)$, and 0 otherwise.

Fuel Map ($M_{\text{fuel}} \in [0, 1]^{H\times W}$): A uniform map where every pixel contains the agent's current normalized fuel level $f_t \in [0, 1]$.

3.3. Action Space Formulation

The agent operates in a discrete action space $\mathcal{A}$ containing four possible movements:

$$\mathcal{A} = \{\, 0, 1, 2, 3 \,\} \quad (2)$$

Each action corresponds to a unit step in one of the four cardinal directions:

$0 \to$ Forward, $\quad 1 \to$ Backward, $\quad 2 \to$ Left, $\quad 3 \to$ Right.

Executing an action $a_t \in \mathcal{A}$ moves the agent by one grid cell in the specified direction, subject to boundary limits and fuel availability.

3.4. Decision Variables

At each time step t, the agent selects an action:

$$a_t \in \mathcal{A} = \{0, 1, 2, 3\}.$$

The sequence of states and actions generated during an episode forms the trajectory:

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_T, a_T).$$

Each action is chosen according to the policy:

$$\mathcal{P} = \pi_\theta(a_t \mid s_t),$$

which denotes the probability that the policy network (parameterized by θ) selects action a_t when the agent is in state s_t.

The value function estimates the expected return from state s_t and is written as:

$$V_\psi(s_t),$$

where ψ represents the parameters of the value network.

3.5. Reward Function

The reward function is designed to guide the agent toward efficient and goal-directed navigation while accounting for terrain difficulty, fuel limitations, and safety constraints. At each time step t, the immediate reward is expressed as:

$$\mathcal{R}(s_t, a_t) = G(s_t) + P(s_t) + F(s_t) + B(s_t) + E(s_t), \quad (3)$$

where each term encourages or penalizes specific behaviours relevant to the fuel-constrained terrain navigation task.

Goal Reward $G(s_t)$: This component provides a large positive reward when the agent reaches a previously unvisited goal. Its purpose is to strongly incentivize visiting all goals:

$$G(s_t) = \begin{cases} \kappa_1, & \text{if a new goal is reached,} \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Progress Reward $P(s_t)$: To ensure consistent movement toward objectives, a shaping reward is provided when the agent reduces its distance to the nearest unvisited goal:

$$P(s_t) = \kappa_2 \left(d(s_{t-1}, g^*) - d(s_t, g^*) \right). \tag{5}$$

This offers smooth guidance even when the agent is far from goals.

Fuel Cost Penalty $F(s_t)$: Terrain elevation affects energy usage. The agent incurs a penalty only when climbing upward:

$$F_{\text{climb}} = \eta_1 \cdot \max(0,\ z_t - z_{t-1}), \qquad F(s_t) = -\kappa_3 F_{\text{climb}}. \tag{6}$$

This discourages unnecessary climbs and promotes fuel-efficient routes.

Boundary Penalty $B(s_t)$: To prevent invalid behaviour such as stepping outside the map, a strong negative reward is applied whenever the agent attempts to move beyond the terrain limits:

$$B(s_t) = \begin{cases} -\kappa_4, & \text{if the agent moves out of bounds,} \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

Exhaustion Penalty $E(s_t)$: Fuel is a hard constraint. If the agent depletes its fuel, a terminal penalty is applied to emphasize the importance of fuel management:

$$E(s_t) = \begin{cases} -\kappa_5, & \text{if fuel is depleted,} \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

3.6. Reward Function Analysis

3.6.1. Continuity of $\mathcal{R}$

The reward function $\mathcal{R}$ consists of both smooth component and event-driven components.

- **Smooth component**, the Progress Reward $P(s_t)$vary continuously with changes in the agent's state. Small changes in position or elevation result in small changes in these terms.
- **Event-triggered components** such as the Goal Reward $G(s_t)$, Boundary Penalty $B(s_t)$, and Exhaustion Penalty $E(s_t)$ introduce discrete jumps when specific conditions (e.g., reaching a new goal or stepping out of bounds) are met.

The overall reward $\mathcal{R}$ is **continuous almost everywhere**, except at critical transition points where intentional discontinuities encode important behavioural constraints.

3.6.2. Differentiability of $\mathcal{R}$

Because of its mixed structure, the differentiability of $\mathcal{R}$ depends on its components.

- **Smooth term** $P(s_t)$ is fully differentiable with respect to the agent's state variables.
- **Event-based terms** ($G(s_t)$, $B(s_t)$, and $E(s_t)$) are inherently non-differentiable at the exact points where the triggering condition activates (e.g., crossing a goal boundary).

Thus, $\mathcal{R}$ is **not globally differentiable**, but has large regions where gradient-based learning remains valid.

3.6.3. Gradients for Non-Differentiable Reward Components in PyTorch

In practice, reinforcement learning systems rely on automatic differentiation frameworks such as **PyTorch** to compute gradients through the reward-driven optimization process. However, several components of $\mathcal{R}$ – especially event-triggered rewards and piecewise terms – are not classically differentiable. PyTorch handles such cases using a set of well-defined fallback rules to ensure that gradient-based learning can continue meaningfully:

- If a function is differentiable at the current point, the exact gradient is used.
- If the function is (locally) convex, PyTorch selects the **sub-gradient of minimum norm**.
- If the function is (locally) concave, the corresponding **super-gradient of minimum norm** is applied.
- If the function is defined but non-differentiable, PyTorch assigns a gradient by **continuity**, choosing an admissible value when multiple options exist.
- If the function is outside its domain (e.g., `log(-1)`, `sqrt(-1)`), PyTorch returns an **arbitrary value** – often NaN – and may raise an error.
- If the operation is not a mathematical function (i.e., non-deterministic), it is marked as **non-differentiable** and will error out if gradients are required.

These mechanisms ensure that even when the reward function $\mathcal{R}$ contains discontinuities or non-differentiable points, PyTorch can still propagate meaningful (sub-)gradient information. Consequently, actor – critic and policy-gradient algorithms remain effective despite the intentional discontinuities built into the reward structure.

3.7. Objective Function

We try to maximize the expected total discounted reward, or "return." This becomes the objective function, $J(\theta)$, that we aim to optimize:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t\mathcal{R}(s_t,a_t)\right] \tag{9}$$

Where:

- τ is a **trajectory** (one full episode) generated by following the policy π_θ.
- $\gamma \in [0,1]$ is the **discount factor**, which determines the importance of future rewards.
- $\mathcal{R}(s_t,a_t)$ is the **immediate reward** received after taking action a_t in state s_t.

The entire training process is dedicated to finding the optimal set of weights θ^* that solves this problem:

$$\theta^* = \arg\max_\theta J(\theta) \tag{10}$$

Directly maximizing $J(\theta)$ is challenging because:

- the expectation depends on unknown environment dynamics,
- trajectory-based estimates exhibit high variance,
- large updates to the policy can cause instability or policy collapse.

To obtain stable and tractable gradients, PPO optimizes a *surrogate* objective rather than $J(\theta)$ directly. The total loss combines the policy loss, value function loss, and an entropy regularization term:

$$\mathcal{L}(\theta,\phi) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_1\mathcal{L}^{\text{VF}}(\phi) - c_2\mathcal{H}[\pi_\theta]. \tag{11}$$

Policy Loss (Clipped Surrogate Objective)

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\,\hat{A}_t,\ \text{clip}\left(r_t(\theta),1-\epsilon,1+\epsilon\right)\hat{A}_t\right)\right], \tag{12}$$

where the probability ratio is

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

and $\hat{A}_t$ is the **Advantage** estimate.

Value Function Loss (Critic)

$$\mathcal{L}^{VF}(\phi) = \mathbb{E}_t\Big[\ell(V_\phi(s_t),\hat{R}_t)\Big], \tag{13}$$

where

$$\hat{R}_t = \hat{A}_t + V_\phi(s_t)$$

The SmoothL1 loss (Huber loss) used in PPO is defined as:

$$\ell(x,y) = \begin{cases} 0.5\,\dfrac{(x-y)^2}{\beta}, & \text{if } |x-y| < \beta, \\ |x-y| - 0.5\beta, & \text{otherwise.} \end{cases}$$

Entropy Bonus (Encouraging Exploration)

$$\mathcal{H}[\pi_\theta] = \mathbb{E}_{s\sim\mathcal{D}}\left[-\sum_{a\in\mathcal{A}}\pi_\theta(a \mid s_t)\log\pi_\theta(a \mid s_t)\right]. \tag{14}$$

3.8. Constraints

3.8.1. Boundary Constraints

The agent must remain within the terrain boundaries:

$$0 \le x_t \le W-1, \qquad 0 \le y_t \le H-1, \qquad \forall t. \tag{15}$$

3.8.2. Movement Constraint

The agent moves exactly one grid cell per time step (Manhattan distance = 1), starting from a fixed initial position:

$$\|s_t - s_{t-1}\|_1 = 1, \qquad \forall t \ge 1. \tag{16}$$

where s_0 is the start position. Fixed 1-unit movement ensures consistent state transitions and enables a constant base fuel cost F_{base} per step.

3.8.3. Fuel Constraints

The agent has limited fuel that depletes with each action, with additional consumption for uphill movement. The cumulative fuel usage up to any time step k must satisfy:

$$\sum_{t=1}^{k}\left[F_{\text{base}} + F_{\text{climb}}(s_{t-1},s_t)\right] \le F_{\text{initial}}, \qquad \forall k \in [1,T]. \tag{17}$$

The elevation-dependent climbing cost is given by:

$$F_{\text{climb}}(s_{t-1},s_t) = \max(0,\ z_t - z_{t-1})\cdot\alpha,$$

where F_{base} is the per-step fuel usage, z_t is the elevation at state s_t, α controls uphill consumption intensity, and F_{initial} is the total fuel available at the start.

4. Proposed Solution

4.1. Stochastic Terrain Generation

The environment is procedurally generated to ensure diverse and non-memorized navigation behaviour. Three key sources of randomness are used:

Perlin Noise Based Elevation Map: A realistic terrain map is generated using Fractal Perlin Noise. This produces smooth but non-uniform elevation patterns, enabling generalizable planning rather than memorization of fixed terrain.

Random Start Position: At the beginning of each episode, the agent's initial state $s_0 = (x_0, y_0)$ is sampled uniformly from the grid. This prevents the policy from overfitting to a single starting configuration.

Random Goal Placement: Goal locations are also sampled randomly for every episode. This creates diverse navigation tasks, requiring the agent to adapt to changing objectives and terrain geometry.

4.2. Network Architecture

The Actor – Critic agent is implemented using a deep neural network tailored for high-dimensional spatial observations generated by the terrain navigation environment. The network adopts a decoupled-encoder design, where the Actor (policy network) and the Critic (value network) each possess independent convolutional encoders. This separation improves representational stability and prevents gradient interference between the two learning objectives of PPO.

Input Representation. Each state is represented as a four-channel spatial tensor,

$$s_t \in \mathbb{R}^{4\times H\times W},$$

with channels corresponding to: terrain height, slope/gradient map, distance-to-goal field, and a fuel/constraint indicator. The spatial resolution satisfies $H = W \in \{128, 256\}$.

Design Overview. The architecture consists of:

- Two independent convolutional encoders (one for Actor, one for Critic),
- Two fully-connected heads (policy logits and value estimation),
- Orthogonal initialization for all convolutional and linear layers,
- LeakyReLU nonlinearities for improved gradient flow.

4.2.1. Convolutional Encoders

Both Actor and Critic encoders share the same structural layout. Given input state s_t, each encoder applies two convolution–pooling stages followed by a flattening operation:

$$\begin{aligned}
&\text{Block 1:} && \mathrm{Conv2D}(4 \to 32,\ 3\times 3,\ \text{padding} = 1),\\
& && \text{LeakyReLU},\\
& && \mathrm{MaxPool2D}(2,2),\\
&\text{Block 2:} && \mathrm{Conv2D}(32 \to 64,\ 3\times 3,\ \text{padding} = 1),\\
& && \text{LeakyReLU},\\
& && \mathrm{MaxPool2D}(2,2),\\
&\text{Flatten:} && z = f_{\text{enc}}(s_t) \in \mathbb{R}^D,
\end{aligned}$$

where

$$D = 64 \cdot \frac{H}{4} \cdot \frac{W}{4}.$$

Separate encoders allow the policy and value functions to learn task-specific representations, avoiding representational collapse.

4.2.2. Actor Network (Policy Head)

The Actor maps encoded features z to logits of a categorical distribution over the discrete action set. The policy head is a three-layer MLP:

$$z \to 512 \to 512 \to A,$$

where A denotes the number of available actions. The resulting logits define the stochastic policy:

$$\pi_\theta(a \mid s_t) = \mathrm{Categorical}(\text{logits}).$$

This distribution is used to sample actions, compute log-probabilities, and evaluate policy entropy during PPO updates.

4.2.3. Critic Network (Value Head)

The Critic estimates the state-value function,

$$V_\phi(s_t) \in \mathbb{R},$$

using an MLP structurally similar to the Actor head:

$$z \to 512 \to 512 \to 1.$$

The Critic output provides a baseline for advantage computation and contributes to the value loss term during optimization.

4.2.4. Forward Pass and Outputs

The forward pass processes the input through both encoders independently:

$$\begin{aligned}
z_A &= f_{\text{enc}}^{(A)}(s_t), \quad \text{logits} = \mathrm{MLP}_{\text{actor}}(z_A),\\
z_C &= f_{\text{enc}}^{(C)}(s_t), \quad V(s_t) = \mathrm{MLP}_{\text{critic}}(z_C).
\end{aligned}$$

The Actor then forms a categorical distribution from the logits to produce the action, log-probability, and entropy.

Action Sampling. An action is sampled as:

$$a_t \sim \mathrm{Categorical}(\text{logits}),$$

accompanied by the log-likelihood $\log \pi_\theta(a_t|s_t)$ and entropy $\mathcal{H}(\pi_\theta)$, which contribute to the PPO objective.

4.3. Optimization Algorithm

To optimize the objective function, we use **Proximal Policy Optimization (PPO)** (Schulman et al., 2017), a policy-gradient reinforcement learning algorithm designed to provide stable and sample-efficient updates. Instead of directly

maximizing $J(\theta)$, PPO performs gradient ascent on a carefully constructed surrogate loss that limits how much the new policy can deviate from the old one during each update.

PPO introduces a *clipped* probability ratio in the policy loss, which forms a soft trust region:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

By restricting $r_t(\theta)$ to lie within the interval

$$1 - \epsilon \;\leq\; r_t(\theta) \;\leq\; 1 + \epsilon,$$

the algorithm prevents excessively large policy updates and avoids catastrophic performance collapse.

During training, PPO alternates between:

- **Policy updates** using the clipped surrogate objective $\mathcal{L}^{\text{CLIP}}(\theta)$,
- **Value function regression** using $\mathcal{L}^{\text{VF}}(\phi)$ to improve state-value estimates,
- **Entropy regularization** $\mathcal{H}[\pi_\theta]$ to encourage sufficient exploration.

This combination yields an optimization procedure that:

- stabilizes training even with noisy or discontinuous rewards,
- avoids destructive policy jumps through the clipping mechanism,
- remains computationally efficient without requiring second-order (Hessian-based) optimization,
- works effectively with large neural networks and stochastic trajectories.

Overall, PPO provides a robust and stable mechanism for improving the policy, making it well-suited for the fuel-constrained terrain navigation problem.

4.4. Update Algorithm

To optimize both the policy and value function parameters, we employ the **AdamW** (Loshchilov & Hutter, 2017) optimizer (Adaptive Moment Estimation with Decoupled Weight Decay). AdamW extends the standard Adam (Kingma, 2014) optimizer by separating the weight decay term fromthe gradient-based update, leading to more stable and better-regularized learning.

At each iteration, the parameter update rule is:

$$\theta_t = \theta_{t-1} - \alpha\lambda\theta_{t-1} - \alpha\,\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{18}$$

where $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected first and second moment estimates of the gradients.

AdamW is particularly suitable for this setting due to several reasons:

- **Decoupled weight decay** provides consistent regularization and prevents overfitting to recent trajectories encountered during PPO training.
- **Adaptive moment estimates** help smooth the inherently noisy gradients arising from stochastic returns and advantage estimates.
- **Improved stability** compared to Adam in non-stationary reinforcement learning environments, where the loss landscape shifts as the policy updates.
- **Better generalization** across varied terrain profiles, especially in procedurally generated environments with high variability.

4.5. Learning Rate Schedule

To ensure stable convergence during training, we employ a **linear learning rate decay**. The learning rate starts at an initial value α_0 and decreases linearly to zero across the training horizon.

A linearly decaying schedule enables **fast early learning** due to a large initial step size and **stable late-stage refinement** as the learning rate reduces. This gradual decay also complements PPO's conservative update rule, leading to smooth and reliable convergence.

5. Results

This section presents the qualitative and quantitative results of the trained fuel-constrained terrain navigation agent. We analyze the terrain, the learned behaviour, and several diagnostics that characterize policy improvement, optimization stability, and value-function accuracy.

5.1. 3D Terrain Visualization

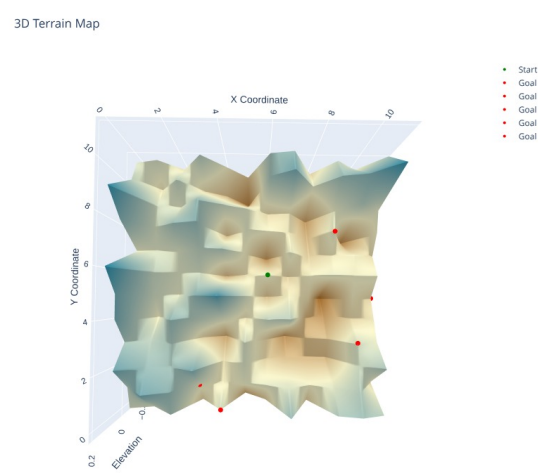

Figure 1. 3D terrain elevation map generated using Perlin noise.

Inference: The visualization highlights significant elevation variations across the map. Despite this complexity, the trained policy is able to navigate uneven terrain while identifying feasible routes to the required goals.

5.2. Best Episode Trajectory

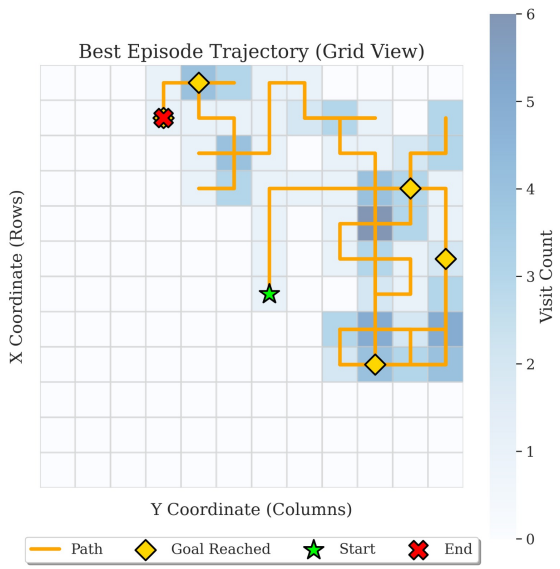

Figure 2. Trajectory of the best-performing episode (grid representation).

Inference: The agent follows an efficient navigation strategy, minimizing unnecessary backtracking and visiting only essential regions of the terrain to reach the goal.

5.3. Cumulative Reward Progression

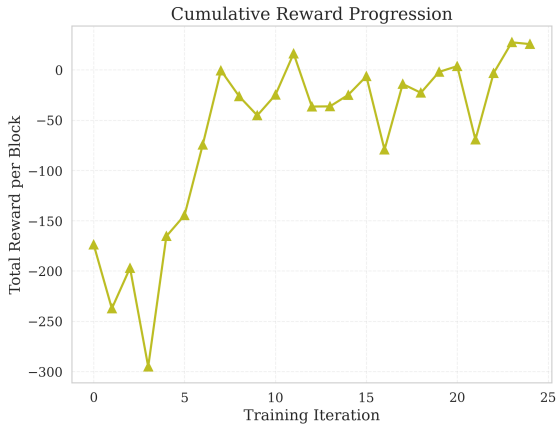

Figure 3. Progression of cumulative reward during training.

Inference: The cumulative reward improves from highly negative values toward zero and beyond, confirming effective learning of the navigation policy.

5.4. Optimization Landscape Roughness

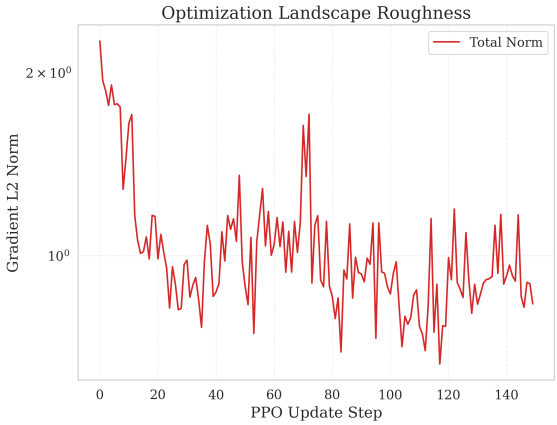

Figure 4. Gradient norm during training.

Inference: The gradient norm decreases significantly, indicating that optimization progresses toward smoother regions of the loss landscape.

5.5. Trust Region Constraint Activation

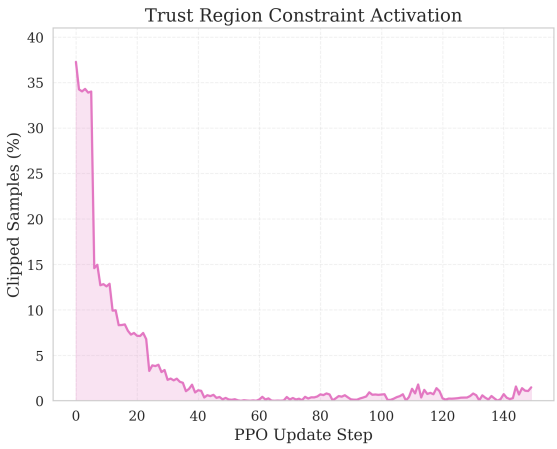

Figure 5. Proportion of clipped samples across training.

Inference: The clipped sample ratio falls near zero after early training, meaning policy updates remain within PPO's trust region.

5.6. Optimization Phase Portrait

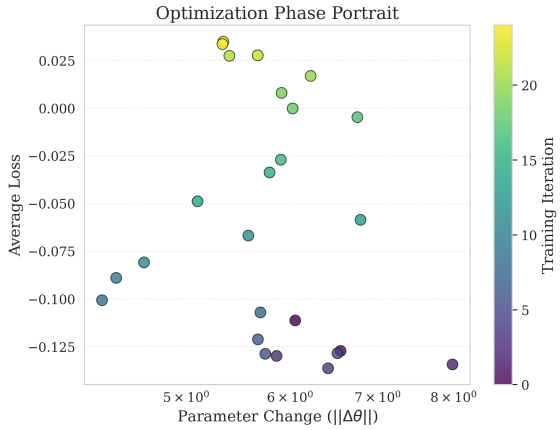

Figure 6. Phase portrait showing parameter update magnitude vs. loss.

Inference: Smaller parameter updates correlate with lower loss values, indicating convergence toward a stable optimum.

5.7. Parameter Space Trajectory

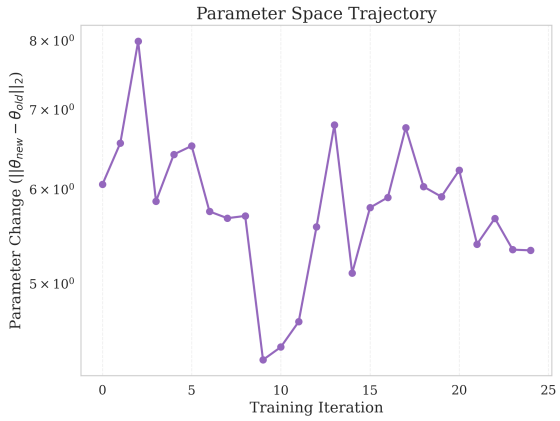

Figure 7. Magnitude of parameter updates over time.

Inference: Parameter update magnitude steadily decreases, reflecting stabilization of the policy.

5.8. Policy Convergence

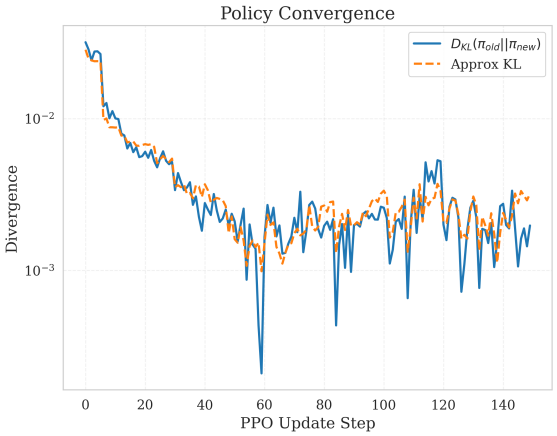

Figure 8. KL divergence between consecutive policies.

Inference: KL divergence falls consistently, demonstrating stable PPO updates that remain close to the previous policy.

5.9. Value Function Quality

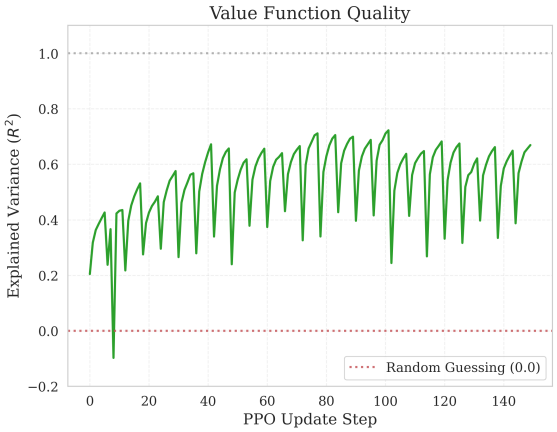

Figure 9. Explained variance of value predictions.

Inference: Explained variance improves to around 0.6, indicating that the critic becomes more accurate in predicting value targets.

5.10. Critic Accuracy: Prediction vs Reality

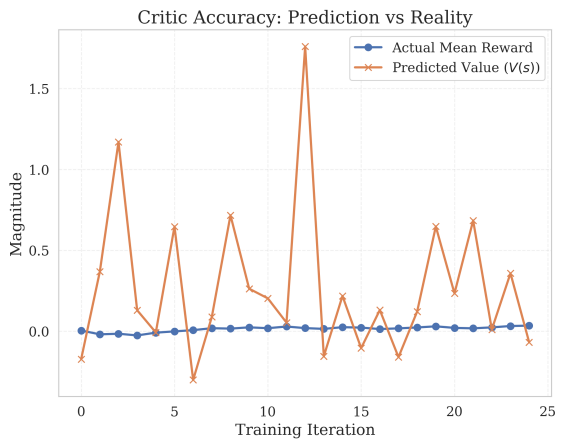

Figure 10. Comparison of critic predictions and actual returns.

Inference: Although noisy, critic predictions track the correct reward trends, showing robustness under stochastic terrain and reward variability.

5.11. Objective Function Decomposition

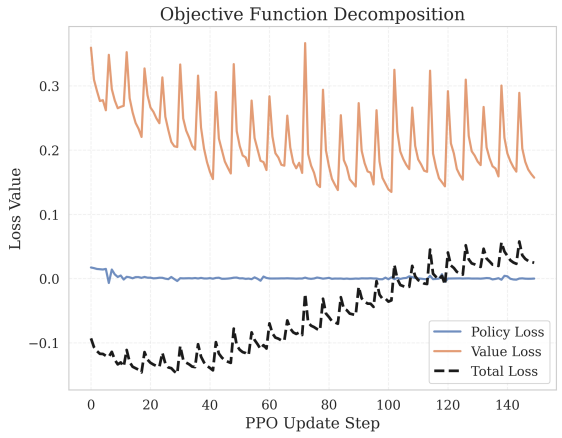

Figure 11. Decomposition of PPO loss components.

Inference: Value loss dominates while policy loss remains stable, consistent with PPO behaviour in sparse-reward navigation tasks.

5.12. Sample Efficiency

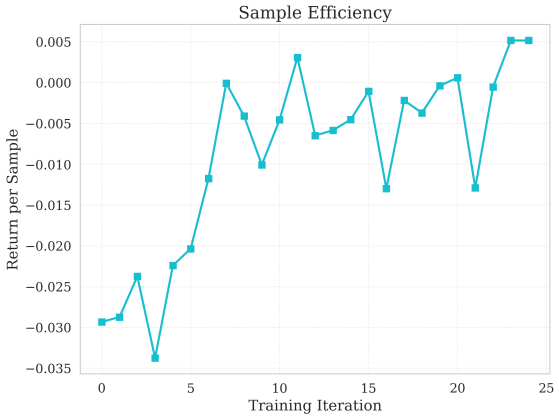

Figure 12. Return per environment step (sample efficiency).

Inference: Return per sample increases steadily, demonstrating that experience is used more effectively as training progresses.

6. Limitations

Despite good empirical performance, the method has several limitations:

- **No strict constraint guarantees:** Boundary and fuel limits are enforced through negative rewards, not hard constraints, so violations are still possible.
- **No guarantee of reaching all goals:** The formulation does not require the agent to visit every goal in an episode; it may skip goals if they are far or fuel-inefficient.
- **Computationally demanding:** PPO training with AdamW and neural policies requires significant compute and benefits strongly from GPU acceleration.
- **Training variability:** Stochastic terrains and random start/goal sampling introduce variance, and multiple runs may be needed for consistent results.

7. Conclusion

This work presented a reinforcement learning approach for fuel-constrained navigation over procedurally generated 3D terrains. Using a structured reward function, PPO for policy optimization, and AdamW for stable parameter updates, the agent learned to navigate uneven terrain, manage fuel efficiently, and reach goals with increasing reliability. Results showed steady improvement in reward, stable policy updates, and better value predictions over time.

Despite these strengths, the method lacks hard guarantees on constraint satisfaction and does not enforce reaching all

goals in every episode. Future work can address these limitations through constrained RL methods or explicit multi-goal requirements.

References

Bollapragada, R., Nocedal, J., Mudigere, D., Shi, H.-J., and Tang, P. T. P. A progressive batching l-bfgs method for machine learning. In *International Conference on Machine Learning*, pp. 620–629. PMLR, 2018.

Hart, P. E., Nilsson, N. J., and Raphael, B. A formal basis for the heuristic determination of minimum cost paths. In *IEEE Transactions on Systems Science and Cybernetics*, volume 4, pp. 100–107, 1968.

Kingma, D. P. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Mirowski, P., Pascanu, R., Viola, F., Soyer, H., Ballard, A. J., Banino, A., Denil, M., Goroshin, R., Sifre, L., Kavukcuoglu, K., et al. Learning to navigate in complex environments. *arXiv preprint arXiv:1611.03673*, 2016.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.

Perlin, K. An image synthesizer. In *ACM SIGGRAPH computer graphics*, volume 19, pp. 287–296. ACM, 1985.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Thrun, S., Burgard, W., and Fox, D. *Probabilistic robotics*. MIT press, 2005.

Zhu, Y., Mottaghi, R., Kolve, E., Lim, J. J., Gupta, A., Fei-Fei, L., and Farhadi, A. Target-driven visual navigation in indoor scenes using deep reinforcement learning. In *2017 IEEE international conference on robotics and automation (ICRA)*, pp. 3357–3364. IEEE, 2017.

8. Appendix

8.1. Stochastic Terrain Generation – Perlin Noise

The terrain is procedurally created using **Fractal Perlin Noise**, enabling generalizable navigation rather than memorization.

$$T(i,j) = \text{Perlin2D}\left(\frac{i}{s}, \frac{j}{s}; o, p, l\right)$$

where $T \in \mathbb{R}^{H \times W}$ is the elevation map, and:

- (i,j): Grid coordinates
- s: Scale
- o: Octaves
- p: Persistence
- l: Lacunarity

Fractal Noise Algorithm

Multiple octaves are combined to form realistic terrain:

$$T(i,j) = \sum_{k=0}^{o-1} A_k \cdot \text{Noise}\left(f_k \cdot \frac{i}{s}, f_k \cdot \frac{j}{s}\right)$$

where:

$$A_k = p^k \quad \text{(amplitude)}, \quad f_k = l^k \quad \text{(frequency)}$$

This composition creates smooth yet detailed terrain variations.

Random Start Position

$$s_0 = (x_0, y_0) \sim \mathcal{U}\{0, \min(H,W) - 1\}^2$$

The agent begins at a uniformly sampled location each episode.

Random Goal Placement

$$g_i = (x_i, y_i) \sim \mathcal{U}\{0, \min(H,W) - 1\}^2, \quad i \in [1, n]$$

$$M_{\text{goals}}(i,j) = \begin{cases} 1.0 & \text{if } g_k = (i,j) \text{ and } g_k \text{ unvisited} \\ 0.0 & \text{otherwise} \end{cases}$$

Stochastic Properties

- Seed-controlled randomness
- Diverse terrains across training episodes
- Encourages generalizable navigation policies

8.2. Proximal Policy Optimisation Training Loop

procedure Proximal Policy Optimizaiton($\pi_{\theta_{\text{old}}}, V_\phi$, epochs K)

Repeat until convergence:
  Collect batch of τ trajectories using $\pi_{\theta_{\text{old}}}$
  Compute **Advantage Estimates** $\hat{A}_t$ using GAE
for $k = 1$ to K training epochs **do**
    Compute **Policy Ratio**: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
    Compute total **Clipped PPO Loss** $\mathcal{L}_{\text{PPO}}(\theta)$
    Compute **Value Function Loss** $\mathcal{L}_{\text{VF}}(\phi)$
    Compute **Entropy Bonus** $\mathcal{L}_{\text{ENT}}(\theta)$
    AdamW($\theta_k, \alpha, \beta_1, \beta_2, \lambda, \epsilon$)
end for
  Update old policy: $\theta_{\text{old}} \leftarrow \theta$
end repeat

8.3. Advantage Function

Advantage measures how much better an action is than the state baseline:

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

Interpretation:

- $A > 0$: action performs better than expected
- $A < 0$: action performs worse than expected

Role in PPO

Advantage is the core learning signal used inside the surrogate objective.

8.4. Generalized Advantage Estimation (GAE) (Schulman et al., 2015)

Definition

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

where the TD residual:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Why GAE?

- **Reduces variance** significantly
- Introduces **minimal bias**
- Makes policy gradient estimates **smoother and more stable**

8.5. Other Update Optimization Algorithms

8.5.1. SGD (Stochastic Gradient Descent)

For an objective

$$Q(w) = \frac{1}{n} \sum_{i=1}^{n} Q_i(w),$$

SGD approximates the full gradient by sampling a single index i, and updates:

$$w := w - \eta \nabla Q_i(w).$$

Why Not Suitable Here:

Gradients are computed from noisy, high-variance sampled rewards and updated under discontinuous terrain transitions. This makes plain SGD too unstable.

8.5.2. ADAM

Adam (Adaptive Moment Estimation) is an optimizer that uses exponential moving averages of the gradient and its squared values to adapt learning rates for each parameter.

$$\theta_t = \theta_{t-1} - \eta \frac{m_t}{\sqrt{v_t} + \varepsilon}.$$

Why Not Suitable Here:
Terrain navigation has very noisy, non-stationary and discontinuous rewards. Adam may over-adapt or become unstable.

8.5.3. L-BFGS

L-BFGS (Limited-memory BFGS) (Bollapragada et al., 2018) is a quasi-Newton optimizer that approximates the inverse Hessian using a small history of past gradients to compute a second-order update direction.

$$\theta_{t+1} = \theta_t - \alpha H_t^{-1} \nabla f(\theta_t),$$

Why Not Suitable Here:
Gradients are noisy and non-stationary, making L-BFGS unstable. Its higher memory cost and requirement for full-batch updates makes it unsuitable for stochastic mini-batch PPO training.

Complexity :

Input: $H = 32, W = 32$	**Batch Size** (B)**:** 64
History (m)**:** 100	**L-BFGS Evals:** ≈ 2.5 avg

1. Network Forward Pass Cost (C_{fwd})

- **Encoders (Actor + Critic):**

$$2 \times [11,520 \times (32 \times 32)] \approx \textbf{23.6 MFLOPs}$$

- **Linear Heads:**

$$2 \times [2 \cdot 4096 \cdot 512] \approx \textbf{8.4 MFLOPs}$$

- **Per-Instance Total:**

$$C_{fwd} \approx 23.6 + 8.4 = \textbf{32 MFLOPs}$$

2. L-BFGS Step Total

$$\text{Cost} \approx n_{evals} \times \underbrace{3 \times B \times C_{fwd}}_{\text{Gradient Calculation (Fwd + Bwd)}}$$

- $n_{evals} \approx 2.5$ (Average evaluations per step)
- Gradient $\approx 3 \times 64 \times 32$ MFLOPs

$$\text{Total} \approx 2.5 \times (6144 \text{ MFLOPs})$$

Computational Load :

Total cost per L-BFGS Step $\approx$ **15.36 GFLOPs**

Memory and Computation Constraints :

- L-BFGS stores m correction pairs (s_k, y_k).
- For a neural network with d parameters, memory requirement scales as $\mathcal{O}(m \cdot d)$.
- In large policy networks (with millions of parameters), this becomes impractical – especially on GPU with limited memory.

- With L-BFGS (History=100) $\approx$ **3.9GB**
- With AdamW $\approx$ **120MB**

8.6. Optimization Algorithm - AdamW Algorithm

procedure AdamW ($\theta_{t-1}, \alpha, \beta_1, \beta_2, \lambda, \epsilon$)
Initialize time step $t \leftarrow 0$, first moment vector $m_0 \leftarrow 0$, second moment vector $v_0 \leftarrow 0$
while parameters θ_t not converged **do**
$t \leftarrow t + 1$
Compute gradient $g_t = \nabla_\theta \mathcal{L}(\theta_{t-1})$
Update biased first moment: $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
Update biased second moment: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
Compute bias-corrected first moment: $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
Compute bias-corrected second moment: $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
Apply weight decay: $\theta_{decay} \leftarrow \theta_{t-1} - \alpha \lambda \theta_{t-1}$
Update parameters: $\theta_t \leftarrow \theta_{decay} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$
end while

8.7. Hyperparameter Table

Hyperparameter	Symbol	Value
Reward Function Parameters		
Goal Reward Weight	κ_1	19
Progress Reward Weight	κ_2	0.2
Fuel Penalty Weight	κ_3	0.1
Boundary Penalty Weight	κ_4	10
Exhaustion Penalty Weight	κ_5	2.5
Climb Cost Scalar ($0.5 \cdot \max$)	η_1	0.5
Entropy Coefficient	λ_e	0.95
Discount Factor	γ	0.95
AdamW Optimizer Parameters		
Learning Rate	α	3×10^{-4}
First Moment Decay	β_1	0.9
Second Moment Decay	β_2	0.999
Numerical Stability Term	ε	10^{-8}
Weight Decay	λ	0.01

Hardware:

- **CPU:** 13th Gen Intel(R) Core(TM) i5-13450HX
- **RAM:** 16GB
- **GPU:** Nvidia RTX 3050 6GB VRAM

8.8. SGD Optimizer

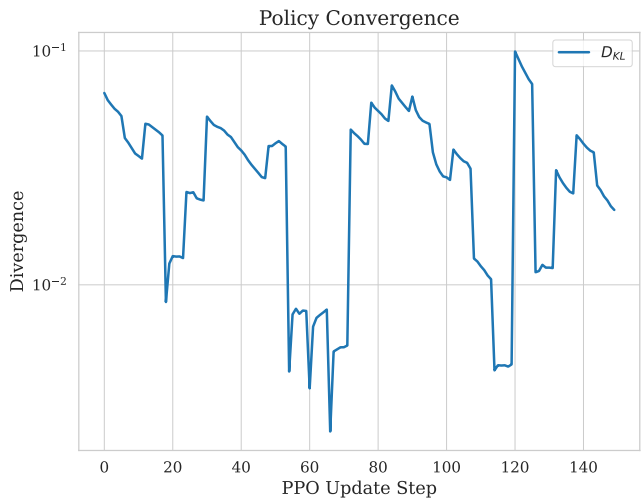

Figure 13. KL Divergence

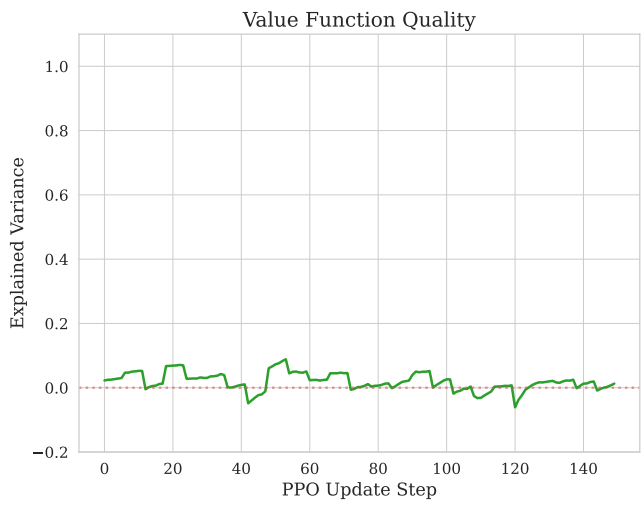

Figure 14. Explained Variance

Figure 15. Parameter Change %

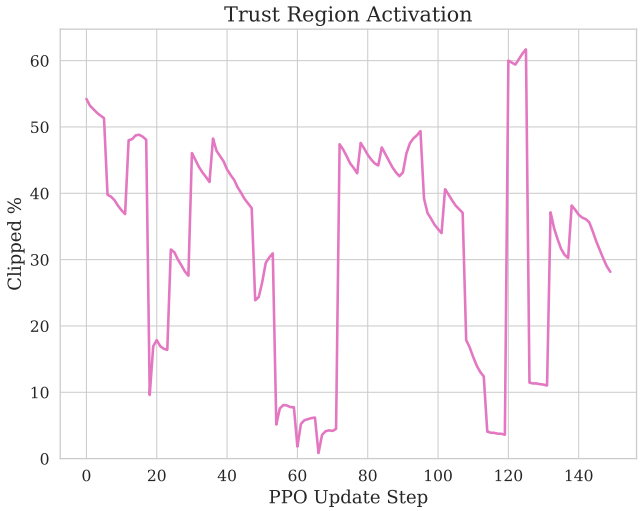

Figure 17. Clipping Ratio

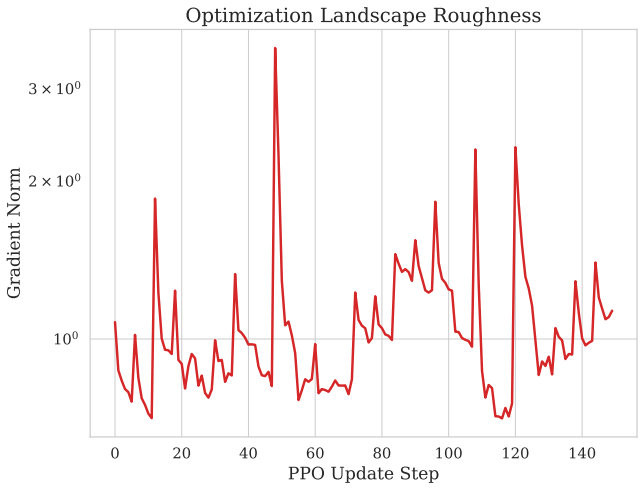

Figure 16. Gradient Norm (L2)

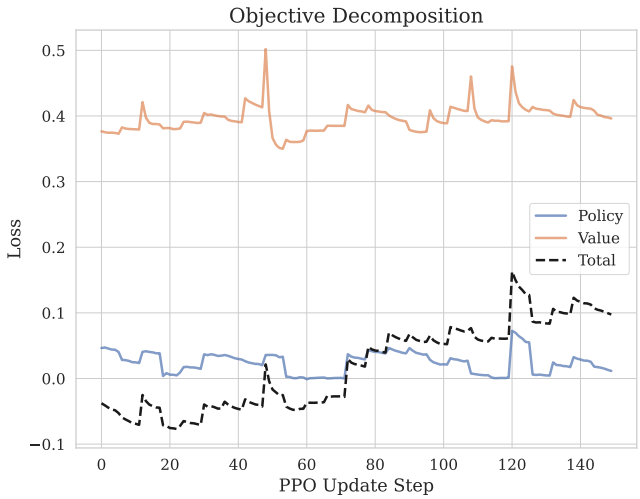

Figure 18. Loss Decomposition

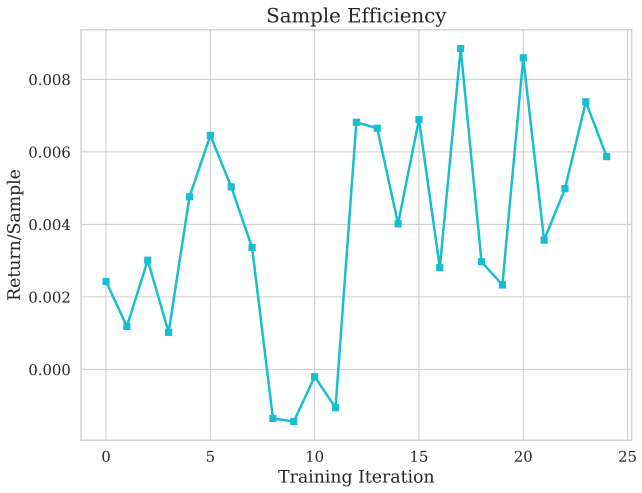

Figure 19. Sample Efficiency

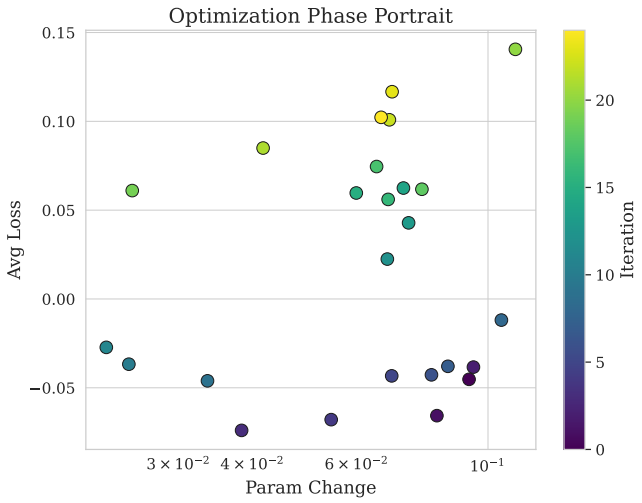

Figure 21. Optimization Phase Potrait

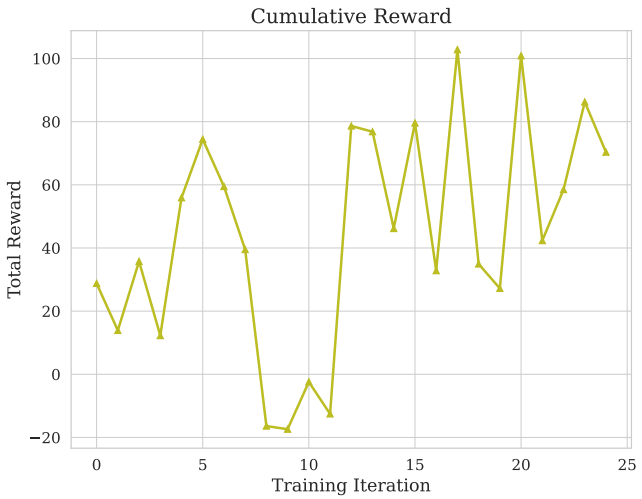

Figure 20. Cumulative Reward

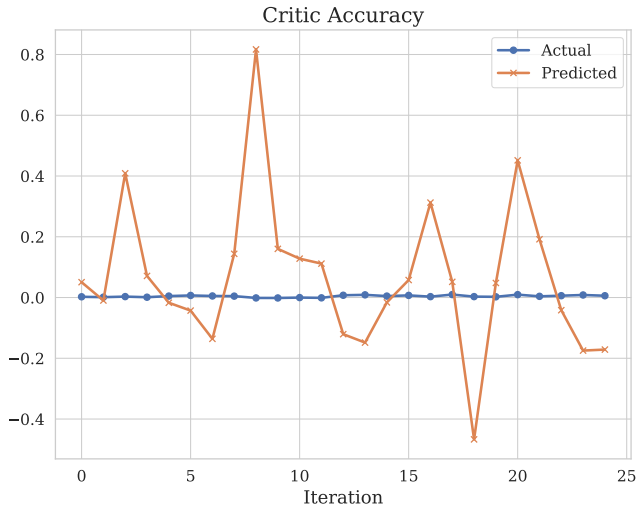

Figure 22. Value Accuracy